

实验 2 指导文档

一、前置步骤：安装 AI 编程工具

✓ Claude Code + GLM-5 安装：

https://ccnk05wgo092.aiforce.cloud/spark/faas/app_4jdqh6vm3jpdg

✓ 智谱 API 获取：<https://open.bigmodel.cn/console/overview>

二、环境配置

1. 终端输入以下命令，依次检查 Python, Node.js, Git, uv 是否已安装：

`python --version`

`node --version`

`git --version`

`uv --version`（若 uv 未安装，直接 `pip install uv` 即可）

2. Git 全局变量设置

`git config --global user.name "xxx"`

`git config --global user.email "xxx"`（邮箱需与你的 GitHub 保持一致）

`git config --global init.defaultBranch main`

`git config --global push.autoSetupRemote true`

`git config --global --list`

GitHub 创建空仓库

3. Spec Kit 与 Insforge 配置

✓ Spec Kit 仓库地址：<https://github.com/github/spec-kit.git>

✓ Insforge 网址：<https://insforge.dev/>（注册并创建项目）

安装 Spec Kit：

`uv tool install specify-cli --from git+https://github.com/github/spec-kit.git`

4. 拓展内容：Skills

✓ Skills 教程：<https://www.bilibili.com/video/BV1G3FNznEiS>

三、项目初始化

1. Spec Kit 与 Git 初始化

创建项目目录并进入

```
specify init .          # 自带 git init
git branch              # 查看分支信息
git remote add origin https://github.com/username/repo-name.git # 关联远程仓库
git add .               # 将所有改动标记为准备提交
git commit -m "Initial commit" # 正式提交（本地）
git push                # 将本地提交记录上传到远程仓库
```

2. 配置 Insforge MCP / Skill

四、Spec Coding

/speckit.constitution 创建一份专注于代码质量、测试标准、用户体验一致性和性能需求的原则文档。此外，本项目中所有文档均使用中文书写。

/speckit.specify 基于 public 文件夹下的 enriched_comments.csv 花园酒店住客评论数据集，搭建一个评论浏览网页，要能按条件筛选、关键词搜索、可以查看图片，并且网页要美观。 # 将切换到 001 分支

/speckit.clarify

/speckit.plan 使用 Next.js 前端、Insforge 后端完成开发，Insforge MCP / Skill 已经配置完成。数据导入要求：

1. 将 enriched_comments.csv 的所有字段导入 Insforge 数据库，表名为 comments
2. 确保数据类型正确（Array 类型字段应设置为 json 类型，publish_date 字段应设置为日期类型）
3. 新增 category1, category2, category3 三个 String 类型字段分别保存 categories 中的小类元素，便于后续检索
4. 新增 star 字段（Integer 类型）将 score 按整数位映射至 1-5 的整数（小于 1 的记为 1）

注：categories 的分类体系如下（数据中仅包含小类名称）：

- 设施类：房间设施、公共设施、餐饮设施
- 服务类：前台服务、客房服务、退房/入住效率
- 位置类：交通便利性、周边配套、景观/朝向
- 价格类：性价比、价格合理性
- 体验类：整体满意度、安静程度、卫生状况

/speckit.tasks /speckit.implement

五、Vibe Coding

体验网站、完成 bug 修复、功能优化

```
npm run dev
```

.....

六、正式提交

检查 .gitignore 是否将 .env, .mcp.json, node_modules, .next 等文件/文件夹忽略

将 001 分支提交到 GitHub

```
git add .
```

```
git commit -m "feat: 完成网页模板开发"
```

```
git push
```

合并 001 分支到 main 并将 main 提交到 GitHub

```
git switch main
```

```
git branch
```

```
git merge branch-name
```

```
git push
```

（可选）删除 001 分支

```
git branch -d branch-name # 删除本地分支
```

```
git push origin -d branch-name # 删除远程分支
```

七、其他事项

1. Tag 管理

标准格式：Major.Minor.Patch（例如 v1.4.2）

- ✓ 主版本 Major: 不兼容的重大重构（如更换底层架构）
- ✓ 次版本 Minor: 新增功能，向下兼容（日常迭代）
- ✓ 修订号 Patch: 修复 Bug，无新功能（紧急修补）

2. Tag 标注与发布（一般标注在 main 分支上）

```
git tag v1.0.0
```

```
git push origin v1.0.0
```

Github 发布 release 版本

3. 克隆 GitHub 仓库并继续开发

`git clone https://github.com/username/repo-name.git` # 下载远程仓库到本地

`cd repo-name` # 进入该目录

`git switch -c branch-name` # 创建并切换至新分支

`git add/commit/push.....` # 在新分支上修改并提交

完成修改后切换至主分支并合并提交

`git switch main`

`git pull` # 同步主分支最新代码

`git merge branch-name -m "合并说明"`

`git push`

`git branch -d branch-name`

`git push origin -d branch-name`

4. （可选）部署到 Vercel

注册并连接 GitHub <https://vercel.com/>

导入项目：Add New Project, Install Vercel, Import 对应项目

根据本地 `.env` 文件添加环境变量

完成部署：Deploy, Continue to Dashboard

修改域名：Domains, Edit（后缀需保留 `.vercel.app`）

/speckit.constitution（示例）

• Constitution 已成功创建。以下是总结：

完成情况

版本：1.0.0（初始版本）

五项核心开发原则：

原则	名称	核心要求
I	数据驱动设计	可视化必须准确反映真实数据，类别严格遵循14个标准小类
II	用户体验优先	全中文界面、响应式设计、页面加载<3秒、问答响应<5秒
III	AI回答可溯源	AI回答必须基于实际评论，展示引用来源，禁止幻觉
IV	性能与可维护性	建立数据库索引、模块化设计、TypeScript类型安全
V	简洁实用	优先使用内置功能、禁止过度工程化

/speckit.specify（示例）

• 完成报告

分支名称: 001-review-dashboard

规格文件: specs/001-review-dashboard/spec.md

质量检查清单: specs/001-review-dashboard/checklists/requirements.md

规格内容摘要

用户故事	优先级	描述
数据可视化看板	P1	7 个必选维度的图表展示，支持筛选联动
评论详情浏览	P2	评论内容和配图展示，支持多条件筛选
智能问答服务	P3	自然语言问答，AI 回答附带评论引用

功能需求: 26 条（FR-001 至 FR-026）

成功标准: 7 条（页面加载 < 3s，问答响应 < 5s，图表联动 < 1s 等）

质量验证: 全部通过，无待澄清项

下一步

规格已就绪，可执行以下命令：

- /speckit.clarify - 如需进一步细化需求
- /speckit.plan - 开始实施规划

/speckit.clarify（示例）

- 我已完成对规格文件的结构化歧义分析。以下是覆盖度评估：

内部覆盖度评估结果：

类别	状态	备注
功能范围与行为	Clear	用户目标、成功标准、范围明确
域与数据模型	Clear	实体、属性、类别已定义
交互与用户流程	Clear	用户故事和验收场景完整
非功能质量属性	Partial	缺少数据规模假设的具体量化
集成与外部依赖	Clear	Insforge/Gemini 依赖已声明
边缘情况与失败处理	Clear	已列出主要边缘情况
约束与权衡	Partial	缺少问答历史是否需要持久化的决策
术语与一致性	Clear	术语使用一致
完成信号	Clear	验收标准可测试

基于分析，我发现有2-3个可能影响实施的歧义点。让我开始逐一澄清。

Question 1: 问答历史持久化

Context: 规格中定义了“问答会话（QA Session）”实体，但未明确用户的问答历史是否需要保存。

What we need to know: 用户的问答历史是否需要持久化存储，以便用户下次访问时可以查看之前的问答记录？

Recommended: Option A - 问答历史是辅助功能，保存历史会增加开发复杂度且对核心数据分析功能无直接贡献。对于 MVP 版本，不保存历史是更务实的选择。

Option	Description
A	不保存，每次会话独立
B	仅保存当前会话（页面刷新后清除）
C	持久化保存，用户可查看历史问答记录
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.

- ---

Question 2: 高质量评论阈值

Context: FR-017 规定“系统必须支持高质量评论推荐（基于 quality_score）”，但未明确什么分数算“高质量”。

What we need to know: quality_score 达到多少分以上的评论应被视为“高质量评论”进行推荐？（数据中 quality_score 范围为 5-10 分，已剔除 ≤4 分）

Recommended: Option B - 选择 8 分作为阈值可以筛选出约 20-30% 的优质评论，既保证质量又有足够数量供用户参考，是平衡质量与数量的合理选择。

Option	Description
A	≥7 分（较宽松，包含更多评论）
B	≥8 分（中等标准，平衡质量与数量）
C	≥9 分（严格标准，仅展示最优质评论）
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.

- ---

Question 3: 问答引用评论数量

Context: FR-021 规定“系统必须展示 AI 回答所依据的原始评论”，但未明确展示多少条引用评论。

What we need to know: 智能问答功能中，AI 回答旁边应展示多少条作为依据的原始评论？

Recommended: Option B - 展示 3-5 条评论可以提供足够的证据支撑 AI 回答的可信度，同时不会让页面过于冗长影响阅读体验。

Option	Description
A	固定展示 3 条最相关评论
B	展示 3-5 条，根据相关性排序
C	展示全部召回评论（可能较多）
Short	Provide a different short answer (<=5 words)

You can reply with the option letter (e.g., "A"), accept the recommendation by saying "yes" or "recommended", or provide your own short answer.

• ---

澄清完成报告

问题数量: 3 个问题已提出并回答

更新的规格文件: <specs/001-review-dashboard/spec.md>

更新的章节:

- Clarifications (新增 Session 2026-01-14)
- Key Entities (QA Session 实体说明更新)
- FR-017 (高质量评论阈值量化)
- FR-021 (引用评论数量明确)

覆盖度总结

类别	状态	说明
功能范围与行为	Clear	用户目标、范围边界明确
域与数据模型	Resolved	问答会话持久化策略已澄清
交互与用户流程	Clear	用户故事完整
非功能质量属性	Resolved	高质量阈值已量化
集成与外部依赖	Clear	Insforge/Gemini 依赖已声明
边缘情况与失败处理	Clear	主要边缘情况已覆盖
约束与权衡	Resolved	引用评论数量已明确
术语与一致性	Clear	术语使用一致
完成信号	Clear	验收标准可测试

无剩余关键歧义。规格已就绪，可进入规划阶段。

建议的下一步

[/speckit.plan](#)

/speckit.plan（示例）

• 规划完成报告

分支: 001-review-dashboard

实施计划文件: specs/001-review-dashboard/plan.md

生成的文档

文件	说明
specs/001-review-dashboard/plan.md	实施计划（技术上下文、宪法检查、项目结构）
specs/001-review-dashboard/research.md	Phase 0 技术研究（图表库、状态管理、数据流设计）
specs/001-review-dashboard/data-model.md	Phase 1 数据模型（Comment 实体、索引设计、TypeScript 类型）
specs/001-review-dashboard/contracts/api.yaml	Phase 1 API 契约（OpenAPI 3.0 规范，5 个端点）
specs/001-review-dashboard/quickstart.md	Phase 1 快速入门指南

宪法检查结果

原则	状态
I. 数据驱动设计	✅ 通过
II. 用户体验优先	✅ 通过
III. AI 回答可溯源	✅ 通过
IV. 性能与可维护性	✅ 通过
V. 简洁实用	✅ 通过

技术决策摘要

- 图表库: Recharts（React 原生，轻量）

- 状态管理: React Context + URL 参数

- 样式方案: Tailwind CSS

- 数据获取: Server Components + API Routes

下一步

执行 /speckit.tasks 命令生成详细任务列表。

注意 gemini 是调用 insforge 中的 AI 模型，而不是使用我自己的 API 调用，检查这一点是否正确

• 已更新所有文档。修正内容：

文件	修正内容
constitution.md	AI 模型改为 google/gemini-3-flash-preview（通过 Insforge AI SDK 调用）
plan.md	移除独立的 Gemini API 依赖，改为 Insforge SDK（含 AI 集成）；移除 gemini.ts 文件
research.md	更新 AI 问答实现方案，添加 Insforge AI SDK 代码示例；更新风险评估
quickstart.md	移除 Gemini API 密钥要求；更新环境配置和故障排除

正确的 AI 调用方式：

```
import { createInsforgeClient } from '@insforge/sdk';

const insforge = createInsforgeClient({ ... });

const completion = await insforge.ai.chat.completions.create({
  model: 'google/gemini-3-flash-preview',
  messages: [...],
  stream: true
});
```

/speckit.tasks（示例）

• 任务生成完成报告

文件路径: `specs/001-review-dashboard/tasks.md`

任务统计

阶段	任务数	说明
Phase 1: Setup	5	项目初始化
Phase 2: Foundational	15	类型定义、数据导入、基础组件
Phase 3: US1 (P1)	17	数据可视化看板
Phase 4: US2 (P2)	10	评论详情浏览
Phase 5: US3 (P3)	10	智能问答服务
Phase 6: Polish	6	优化和收尾
总计	63	

用户故事任务分布

用户故事	优先级	任务数	独立测试标准
US1 - 数据可视化看板	P1	17	访问首页，7 个图表渲染，筛选联动
US2 - 评论详情浏览	P2	10	评论列表展示，筛选功能，图片查看
US3 - 智能问答服务	P3	10	输入问题，获得 AI 回答和引用评论

并行机会

- Phase 2: 4 个类型文件并行，3 个 UI 组件并行

- Phase 3: 7 个图表组件并行，2 个筛选组件并行

- Phase 4: 2 个评论组件并行

- Phase 5: 2 个问答组件并行

MVP 范围

建议 MVP 只完成 Phase 1 + Phase 2 + Phase 3 (US1)，共 37 个任务，即可交付可用的数据可视化看板。

格式验证

✔

所有任务均符合 - [] [TaskID] [P?] [Story?] Description with file path 格式

下一步

执行 `/speckit.implement` 开始实现任务，或手动按任务列表顺序执行。

/speckit.implement（示例）

- Checklist 状态检查

Checklist	Total	Completed	Incomplete	Status
requirements.md	16	16	0	✓ PASS

所有 checklist 已完成，继续执行实现。

- 现在开始执行 Phase 1: Setup。首先使用 Insforge MCP 下载 Next.js 模板。

- Phase 1 完成 

现在开始 Phase 2: Foundational。首先并行创建 TypeScript 类型文件。

直至全部完成.....